Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For



Docker Fundamentals for Beginners

Course Code: C1285

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	3 October 2026	First version — step-by-step guide to all 19 Docker & Kubernetes labs for the one-day course.	Tertiary Infotech Academy Pte Ltd

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Welcome! This guide walks you command-by-command through every hands-on lab in the course **Docker Fundamentals for Beginners** (Course Code: C1285). Over one day you build and deploy one real application — **TaskBoard**, a Flask task tracker — from a single container, to a multi-service Docker Compose stack, to a scalable Kubernetes deployment.

Work through the labs in order: each one builds on the last. Whenever you see a **Test it** box, stop and confirm the result before moving on. All labs also run in the browser on KillerCoda — the link is at the top of each lab.

Note: Course flow at a glance — **Morning (Docker):** commands, Dockerfile & build, CMD vs ENTRYPOINT, volumes, networking, environment config, Docker Hub (Labs 1–9). **Afternoon (Docker Compose & Kubernetes): Docker Compose (Labs 10–12),** Pods, Namespaces, Deployments, rollouts, Services, storage, Jobs & CronJobs (Labs 13–19).

---

0. Before You Start — Setup & Prerequisites

0.1 What you need

Tool	Used for	Where to get it
Docker Engine / Desktop	Building & running containers (all Docker labs)	docker.com (or use KillerCoda in the browser)
Docker Compose	Multi-service labs 10–12	Included with Docker Desktop / Docker Engine
kubectl + a cluster	All Kubernetes labs	KillerCoda playground, or minikube/kind locally
Docker Hub account	Pushing your image (Lab 9)	hub.docker.com — free

0.2 Two ways to run every lab

Option A — KillerCoda (fastest). Each lab header has a KillerCoda link; the commands run in a browser terminal with Docker and Kubernetes pre-installed — nothing to install.

Option B — Local. Install Docker Desktop (gives you Docker + Compose) and a local Kubernetes (minikube, kind, or Docker Desktop's built-in Kubernetes) for the Kubernetes labs.

0.3 Get the lab files

Every lab folder under **`labs/`** is self-contained — it holds the `lab.md` steps plus the working files (app code, `Dockerfile`, `docker-compose.yml`, Kubernetes YAML). The shared sample app lives in **`labs/app/`**.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: **GitHub repo:** https://github.com/tertiarycourses/C1285-Application-Integration-with-Docker-and-Kubernetes · clone it or use **Code → Download ZIP**, then `cd` into each lab folder as you go.

Lab 1 — Docker Commands — Run & Manage Containers

Folder: `labs/lab01-docker-commands/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

Goal

Get comfortable with the core Docker commands by running a real web server (nginx) and managing its whole lifecycle: start it, list it, read its logs, run a command inside it, then stop and remove it.

What you'll build

Run and manage an nginx web server with docker run / ps / logs / exec / stop / rm.

Part A — Run a web server in the background

`nginx` is a real, production web server. Start it detached (`-d`), give it a name, and publish its port 80 to port 8080 on the host:

```
docker run -d --name web -p 8080:80 nginx:latest
docker ps                       # the container is running
curl http://localhost:8080      # nginx serves its welcome page
```

Part B — Inspect a running container

Read its logs, then run a command *inside* the container to see its hostname:

```
docker logs web
docker exec web cat /etc/hostname
docker exec -it web /bin/bash   # open an interactive shell, then 'exit'
```

Part C — Stop, restart and clean up

```
docker stop web
docker ps -a                    # shows stopped containers too
docker start web
docker rm -f web                # force-remove (stops if running)
```

Note: `docker ps` lists only **running** containers; `docker ps -a` lists **all** containers including stopped ones.

✅ **Test it:** Open http://localhost:8080 and confirm the nginx welcome page loads while the container is running, and that `docker ps` no longer lists `web` after `docker rm -f web`.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 2 — Images & Inspecting Containers — pull, cp, inspect

Folder: `labs/lab02-images-and-inspect/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

Goal

Work with images and look inside containers. You will pull images, copy a file out of a running container with `docker cp`, and inspect container details.

What you'll build

Pull images, copy a served file out of nginx with docker cp, and inspect a container.

Part A — Work with images

```
docker pull nginx:latest
docker images                  # list local images
docker run -d --name web -p 8080:80 nginx:latest
```

Part B — Copy a file out of the container (docker cp)

nginx serves `/usr/share/nginx/html/index.html`. Copy it to the host, edit it, and copy it back to change the live page — no rebuild needed:

```
docker cp web:/usr/share/nginx/html/index.html ./index.html
echo '<h1>Hello from TaskBoard class!</h1>' > index.html
docker cp ./index.html web:/usr/share/nginx/html/index.html
curl http://localhost:8080     # shows your new page
```

Part C — Inspect the container

```
docker inspect web | grep -i ipaddress
docker stats --no-stream web   # CPU / memory snapshot
docker rm -f web
```

Note: `docker cp` copies files between a container and the host in either direction — handy for grabbing logs or config out of a running container.

✅ **Test it:** After copying the edited file back, `curl http://localhost:8080` should show your custom heading instead of the default nginx page.

Lab 3 — Build the TaskBoard Image with a Dockerfile

Folder: `labs/lab03-build-image/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

Goal

Package a real application — the TaskBoard Flask web app — into your own Docker image using a Dockerfile, then run it as a container. This image is reused by several later labs.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

What you'll build

Write a Dockerfile for the TaskBoard Flask app, build it, and run the container.

What's in the folder

`app.py` (the TaskBoard Flask app), `templates/index.html` (the board UI), `requirements.txt`, and a `Dockerfile`:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Each line is a cached image **layer**. Notice `requirements.txt` is copied and installed *before* the app code, so editing `app.py` doesn't re-run `pip install`.

Build the image

```
docker build -t taskboard:1.0 .
docker images | grep taskboard
```

Run the container

```
docker run -d --name taskboard -p 8080:5000 taskboard:1.0
curl http://localhost:8080/health
# open http://localhost:8080 and add a few tasks
```

Watch the build cache

Edit `app.py` (e.g. change `APP_TITLE` default) and rebuild — only the layers from `COPY . .` onward rebuild; the `pip install` layer is reused from cache:

```
docker build -t taskboard:1.0 .
docker rm -f taskboard
```

Note: Tag images with a name **and** version (`taskboard:1.0`) so you can roll forward/back later — exactly what you'll do in the Kubernetes rollout lab.

✅ **Test it:** `curl http://localhost:8080/health` returns `{"status": "ok", ...}` and the board at http://localhost:8080 lets you add a task that appears in the list.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 4 — Dockerfile Best Practices — .dockerignore & layer caching

Folder: `labs/lab04-dockerfile-best-practices/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

Goal

Make the TaskBoard image smaller and faster to build. You'll add a `.dockerignore`, order instructions for maximum cache reuse, and compare image sizes.

What you'll build

Add a .dockerignore and cache-friendly layer ordering to slim the TaskBoard build.

Why .dockerignore

`COPY . .` copies everything in the folder into the image — including `.git/`, caches and local `data/`. A `.dockerignore` keeps them out, shrinking the image and the build context:

```
__pycache__/
*.pyc
data/
.git/
.env
*.md
```

Cache-friendly ordering

Dependencies change rarely; source changes often. Copy and install `requirements.txt` **before** copying the source so the dependency layer is cached across code edits (this is already done in our Dockerfile):

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

Build and compare

```
docker build -t taskboard:slim .
docker images taskboard
# edit app.py, rebuild, and watch most layers say 'CACHED'
docker build -t taskboard:slim .
```

Note: `python:3.11-slim` is already a smaller base than `python:3.11`. For even smaller images, multi-stage builds copy only the artifacts you need into a fresh final stage.

✅ **Test it:** The second `docker build` completes in seconds with most steps showing `CACHED`, and `docker images taskboard` shows the image built from the slim base.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 5 — CMD vs ENTRYPOINT — the taskboard-cli tool

Folder: `labs/lab05-cmd-entrypoint/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

Goal

Understand the difference between CMD and ENTRYPOINT by packaging a real command-line tool — `taskboard-cli` — that lists and adds tasks. ENTRYPOINT fixes the program; CMD supplies the default arguments you can override at runtime.

What you'll build

Package taskboard-cli so ENTRYPOINT is fixed and CMD/args are overridable.

The Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY cli.py .
# ENTRYPOINT fixes the executable; CMD is the default argument (overridable).
ENTRYPOINT ["python", "cli.py"]
CMD ["list"]
```

`ENTRYPOINT` is the executable that always runs. `CMD` is the **default argument** — here `list` — which is replaced by anything you pass on `docker run`.

Build it

```
docker build -t taskboard-cli .
```

CMD is the default; run args override it

```
docker run --rm taskboard-cli                 # runs the default: list
docker run --rm taskboard-cli add "Buy milk"   # overrides CMD with: add ...
docker run --rm taskboard-cli add "Ship release"
docker run --rm taskboard-cli list
```

ENTRYPOINT vs CMD — the contrast

	CMD only	ENTRYPOINT + CMD
Dockerfile	CMD ["python","cli.py","list"]	ENTRYPOINT ["python","cli.py"] CMD ["list"]
`docker run img`	runs list	runs list
`docker run img add x`	tries to run `add` as a command (error)	runs `cli.py add x` ✔

Note: Use **ENTRYPOINT** when the image *is* a specific tool and arguments vary (like our CLI). Use **CMD** alone when you want an easily replaceable default command. Both use the JSON *exec form* `["a","b"]` to avoid an extra shell.

✅ **Test it:** `docker run --rm taskboard-cli` prints the task list, and `docker run --rm taskboard-cli add "Demo"` adds a task — the same image, different CMD arguments.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 6 — Docker Storage — Named Volumes & Bind Mounts

Folder: `labs/lab06-volumes/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-02-docker-storage

Goal

Make TaskBoard's data survive container removal. You'll persist its tasks.json in a **named volume**, prove the data outlives the container, then use a **bind mount** to edit files on the host and see them live in the container.

What you'll build

Persist TaskBoard tasks in a named volume; live-edit host files via a bind mount.

The problem: container data is ephemeral

By default everything written inside a container lives in its writable layer and is **lost** when the container is removed. TaskBoard writes its tasks to `DATA_DIR` — point that at a mounted volume and the data persists.

Part A — Named volume (persistent app data)

```
docker volume create taskboard-data
docker run -d --name tb -p 8080:5000 \
  -e DATA_DIR=/data -v taskboard-data:/data taskboard:1.0
# add a few tasks at http://localhost:8080, then:
docker rm -f tb
```

Start a **brand-new** container on the **same volume** — your tasks are still there:

```
docker run -d --name tb2 -p 8080:5000 \
  -e DATA_DIR=/data -v taskboard-data:/data taskboard:1.0
curl http://localhost:8080/api/tasks   # the tasks you added are back
docker rm -f tb2
```

Part B — Bind mount (live host files)

A bind mount maps a host folder straight into the container. Edit on the host, see it instantly inside — great for development:

```
mkdir -p data && echo '[]' > data/tasks.json
docker run -d --name tb3 -p 8080:5000 \
  -e DATA_DIR=/data -v $(pwd)/data:/data taskboard:1.0
# add tasks in the browser, then read them straight off the host:
cat data/tasks.json
docker rm -f tb3
```

	Named volume	Bind mount
Syntax	-v taskboard-data:/data	-v $(pwd)/data:/data
Managed by	Docker	You (host path)
Best for	production data, databases	local development, live edits

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Named volumes are stored under Docker (`/var/lib/docker/volumes`) and are the right choice for databases; bind mounts tie you to an exact host path but are perfect for editing code live.

✅ **Test it:** After removing `tb` and starting `tb2` on the same named volume, `curl http://localhost:8080/api/tasks` returns the tasks you added earlier — they survived container deletion.

Lab 7 — Docker Networking — Custom Bridge, DNS & Port Mapping

Folder: `labs/lab07-networking/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-03-docker-networking

Goal

Connect two containers the way real apps do: TaskBoard reaching a Redis cache by name over a custom bridge network. You'll see why a custom network (with built-in DNS) beats the default bridge, then publish the app to the host with port mapping.

What you'll build

TaskBoard reaches Redis by DNS name on a custom network; publish ports to the host.

Why a custom network

On the **default** bridge, containers can only reach each other by IP address (which changes on restart). On a **custom** bridge network, Docker provides automatic DNS, so a container can reach another by its **name** — exactly what a web app needs to find its database or cache.

Part A — Create a network and a Redis cache

```
docker network create tasknet
docker run -d --name redis --network tasknet redis:7-alpine
```

Part B — Run TaskBoard on the same network

TaskBoard reads `REDIS_HOST` to find the cache. Set it to the **container name** `redis` — DNS on `tasknet` resolves it automatically:

```
docker run -d --name tb --network tasknet -p 8080:5000 \
  -e REDIS_HOST=redis taskboard:1.0
curl http://localhost:8080      # 'visits' counter increments on each load
curl http://localhost:8080
```

Prove the DNS name resolves from inside the app container:

```
docker exec tb getent hosts redis   # resolves to redis's IP on tasknet
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Part C — Port mapping (host access)

`-p 8080:5000` maps host port 8080 to the container's 5000. Run a second instance on a different host port — same image, two endpoints:

```
docker run -d --name tb-b --network tasknet -p 8081:5000 \
  -e REDIS_HOST=redis taskboard:1.0
curl http://localhost:8081/health
docker rm -f tb tb-b redis
docker network rm tasknet
```

Note: Because both TaskBoard instances share one Redis on the network, the visit counter is shared across them — your first taste of stateful, multi-container apps (which Compose and Kubernetes automate).

✅ **Test it:** Reloading http://localhost:8080 increases the `visits` count (served from Redis), and `docker exec tb getent hosts redis` resolves the `redis` name to an IP on the custom network.

Lab 8 — Configuration with Environment Variables

Folder: `labs/lab08-env-vars/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-04-docker-config

Goal

Configure the SAME TaskBoard image for different environments without rebuilding. You'll set defaults with ENV in the Dockerfile, override per-container with `-e`, and load many values at once from an `--env-file`.

What you'll build

Configure TaskBoard (APP_ENV, APP_TITLE) via ENV, -e and --env-file — no rebuild.

Three ways to pass configuration

TaskBoard reads `APP_ENV` and `APP_TITLE` at runtime. The image already ships sensible defaults; you override them per environment.

1. Defaults baked into the image (ENV)

```
docker run -d --name tb -p 8080:5000 taskboard:1.0
curl http://localhost:8080/health   # "env": "development"
docker rm -f tb
```

2. Override per container with -e

```
docker run -d --name tb -p 8080:5000 \
  -e APP_ENV=staging -e 'APP_TITLE=TaskBoard (Staging)' taskboard:1.0
curl http://localhost:8080/health   # "env": "staging"
docker rm -f tb
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

3. Load many values from an --env-file

Keep environment config in a file (`.env`) and load it all at once:

```
cat .env
docker run -d --name tb -p 8080:5000 --env-file .env taskboard:1.0
curl http://localhost:8080/health   # "env": "production"
docker rm -f tb
```

Note: Never bake secrets into an image. Pass them at runtime with `-e` / `--env-file` (and in Kubernetes, with ConfigMaps and Secrets).

✅ **Test it:** The `/health` endpoint reports `development`, then `staging`, then `production` as you change only the environment variables — the image is never rebuilt.

Lab 9 — Sharing Images — Push to Docker Hub

Folder: `labs/lab09-docker-hub/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

Goal

Publish your TaskBoard image to Docker Hub so anyone (and any Kubernetes cluster) can pull and run it. You'll log in, tag the image under your account, push it, then pull and run it as if you were a new user.

What you'll build

Tag, push and pull the TaskBoard image via Docker Hub.

Registry vs repository

A **registry** (Docker Hub) hosts **repositories**; each repository holds the tagged versions of one image. Public images like `nginx` and `redis` live there too.

Log in and tag

Replace `<user>` with your Docker Hub username:

```
docker login
docker tag taskboard:1.0 <user>/taskboard:1.0
```

Push

```
docker push <user>/taskboard:1.0
# browse to https://hub.docker.com/r/<user>/taskboard to see it
```

Pull & run as a new user

```
docker rmi <user>/taskboard:1.0          # remove the local copy
docker run -d --name tb -p 8080:5000 <user>/taskboard:1.0
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
curl http://localhost:8080/health
docker rm -f tb
docker logout
```

Note: This is exactly how Kubernetes gets your app this afternoon: the cluster pulls your image from a registry by `name:tag`. Always push a versioned tag, not just `latest`.

✅ **Test it:** Your image appears at hub.docker.com under `<user>/taskboard`, and after removing the local image you can `docker run` it straight from the registry.

Lab 10 — Docker Compose — Single Service

Folder: `labs/lab10-compose-single/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-05-docker-compose

Goal

Replace long `docker run` commands with a single declarative file. You'll define TaskBoard in a docker-compose.yml and manage it with the compose lifecycle: pull, up, ps, logs, down.

What you'll build

Define TaskBoard in docker-compose.yml; manage it with pull / up / ps / logs / down.

The compose file

```
services:
  web:
    build: .            # build from the Dockerfile in this folder
    ports:
      - "8080:5000"     # host:container
    environment:
      - APP_ENV=development
      - APP_TITLE=TaskBoard (Compose)
```

The Compose lifecycle

```
docker compose pull        # pull any pre-built images (none to build here)
docker compose up -d       # build if needed and start in the background
docker compose ps          # service status
docker compose logs web    # logs for the web service
curl http://localhost:8080/health
```

Tear down

```
docker compose down        # stop and remove containers + network
docker compose down -v     # also remove named volumes (full reset)
```

Note: `docker compose up -d` is idempotent — re-run it after editing the file and Compose only changes what's needed. One file replaces a page of `docker run` flags.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

✅ **Test it:** `docker compose up -d` starts the service and http://localhost:8080 serves TaskBoard titled 'TaskBoard (Compose)'; `docker compose down` removes it cleanly.

Lab 11 — Docker Compose — Multi-Service (Web + Redis)

Folder: `labs/lab11-compose-redis/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-05-docker-compose

Goal

Add a second service. Compose puts both containers on one network and gives each a DNS name equal to its service name, so TaskBoard reaches Redis at host `redis` with zero manual networking. The visit counter is now shared and survives restarts.

What you'll build

Run TaskBoard + Redis with Compose; services find each other by name automatically.

Two services, one file

```
services:
  web:
    build: .
    ports: ["8080:5000"]
    environment:
      - REDIS_HOST=redis     # <-- the other service's name
    depends_on: [redis]
  redis:
    image: redis:7-alpine
    volumes: [redis-data:/data]
volumes:
  redis-data:
```

Compose creates a network automatically and registers each service under its name, so `REDIS_HOST=redis` just works — no IPs, no `docker network create`.

Run the full lifecycle

```
docker compose pull        # pulls redis:7-alpine
docker compose up -d
docker compose ps
curl http://localhost:8080 && curl http://localhost:8080  # visits go up
```

Prove persistence, then tear down

```
docker compose restart web
curl http://localhost:8080   # counter kept (it lives in redis-data)
docker compose down          # keep the volume
docker compose down -v       # remove the redis-data volume too
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: `depends_on` controls **start order**, not readiness — the next lab adds a **health check** so the web service waits until the database is truly ready.

✅ **Test it:** Reloading http://localhost:8080 increments the visit counter, and the count survives `docker compose restart web` because it is stored in the `redis-data` volume.

Lab 12 — Docker Compose — Full-Stack (Web + PostgreSQL + Redis)

Folder: `labs/lab12-compose-fullstack/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day1-05-docker-compose

Goal

Build the complete TaskBoard stack: the web app backed by PostgreSQL for tasks and Redis for the visit counter, with a health check so the app starts only once the database is ready. This is the architecture you'll deploy to Kubernetes later today.

What you'll build

Run TaskBoard + Postgres + Redis with healthchecks and condition-based depends_on.

The full stack

Three services: `web` (TaskBoard), `db` (PostgreSQL, storing tasks), and `redis` (visit counter). The web service waits for the database's **health check** to pass before it starts, so it never crashes on a not-yet-ready database.

```
services:
  web:
    build: .
    ports: ["8080:5000"]
    environment:
      - DATABASE_URL=postgresql://taskboard:secret@db:5432/taskboard
      - REDIS_HOST=redis
    depends_on:
      db: { condition: service_healthy }
      redis: { condition: service_started }
  db:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U taskboard"]
      interval: 5s
      retries: 5
  redis: { image: redis:7-alpine }
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Run the lifecycle

```
docker compose pull        # pulls postgres + redis
docker compose up -d        # builds web, waits for db health, then starts web
docker compose ps           # db shows (healthy)
docker compose logs web
```

Verify tasks persist in Postgres

```
# add tasks at http://localhost:8080, then prove they're in the DB:
docker compose exec db psql -U taskboard -c 'SELECT id, text, done FROM tasks;'
docker compose restart web
curl http://localhost:8080/api/tasks   # tasks still there (in Postgres)
```

Tear down

```
docker compose down         # stop & remove containers + network
docker compose down -v      # also drop the pgdata volume
```

Note: This web + database + cache shape is the canonical cloud app. Later today you'll deploy this very app to Kubernetes — as Pods, a Deployment, a Service, and persistent storage.

✅ **Test it:** `docker compose ps` shows `db` as `(healthy)`, tasks added in the browser appear in `SELECT * FROM tasks`, and they survive `docker compose restart web`.

Lab 13 — Kubernetes Pods — Imperative & Declarative

Folder: `labs/lab13-pods/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day2-01-k8s-pods-namespaces

Goal

Meet the smallest deployable unit in Kubernetes — the Pod. You'll create one imperatively with a single command, inspect it, then create one declaratively from a YAML file (the approach you'll use for everything afterwards).

What you'll build

Create, inspect and delete a Pod both imperatively and from a pod.yaml manifest.

Part A — Imperative (one command)

```
kubectl run web --image=nginx
kubectl get pods
kubectl get pods -o wide          # node + Pod IP
kubectl describe pod web          # events, container, volumes
```

Look inside the Pod, then remove it:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
kubectl logs web
kubectl exec web -- cat /etc/hostname
kubectl exec -it web -- /bin/sh   # interactive shell, then 'exit'
kubectl delete pod web
```

Part B — Declarative (pod.yaml)

Real work is declarative: you describe the desired state in YAML and apply it.

```
apiVersion: v1
kind: Pod
metadata:
  name: taskboard
  labels:
    app: taskboard
spec:
  containers:
    - name: web
      image: nginx          # swap for <user>/taskboard:1.0 to run your own
image
      ports:
        - containerPort: 80

kubectl apply -f pod.yaml
kubectl get pods
kubectl describe pod taskboard
kubectl delete -f pod.yaml
```

Note: Pods are **ephemeral** — if a Pod dies it is not recreated. That's why you almost always run Pods through a **Deployment** (Lab 15), not on their own.

✅ **Test it:** `kubectl get pods` shows the `taskboard` Pod as `Running` after `kubectl apply`, and it disappears after `kubectl delete -f pod.yaml`.

Lab 14 — Kubernetes Namespaces — Environment Isolation

Folder: `labs/lab14-namespaces/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day2-01-k8s-pods-namespaces

Goal

Use namespaces to keep environments apart inside one cluster. You'll create a `dev` namespace, run a Pod in it, and see how resources are isolated by namespace.

What you'll build

Create a dev namespace, run a Pod inside it, and list resources per namespace.

List and create namespaces

```
kubectl get namespaces
kubectl create namespace dev
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Run a Pod inside a namespace

```
kubectl run web --image=nginx -n dev
kubectl get pods -n dev
kubectl get pods                  # default ns: not shown here
kubectl get pods --all-namespaces
```

Clean up the whole namespace

Deleting a namespace removes **everything** inside it — a fast way to tear down an environment:

```
kubectl delete namespace dev
```

Note: Namespaces are perfect for separating `dev` / `staging` / `prod` (or per-team quotas) on a single shared cluster.

✅ **Test it:** `kubectl get pods -n dev` lists the Pod while it exists, and `kubectl delete namespace dev` removes the namespace and its Pod together.

Lab 15 — Deployments — Scaling & Self-Healing

Folder: `labs/lab15-deployments/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day2-02-k8s-deployments

Goal

Run TaskBoard the production way — as a Deployment that keeps a desired number of replicas alive. You'll scale it, watch it self-heal when a Pod is deleted, and manage it both imperatively and from YAML.

What you'll build

Create a Deployment, scale it, and watch Kubernetes self-heal a deleted Pod.

Part A — Create and scale (imperative)

```
kubectl create deployment taskboard --image=nginx
kubectl get deployments
kubectl get pods
kubectl scale deployment taskboard --replicas=3
kubectl get pods               # now 3 Pods
```

Part B — Watch it self-heal

Delete one Pod and watch the Deployment's ReplicaSet immediately create a replacement to restore the desired count:

```
kubectl get pods
kubectl delete pod <one-pod-name>
kubectl get pods               # a new Pod is already being created
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Part C — Declarative (deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taskboard
spec:
  replicas: 3
  selector:
    matchLabels:
      app: taskboard
  template:
    metadata:
      labels:
        app: taskboard
    spec:
      containers:
        - name: web
          image: nginx
          ports:
            - containerPort: 80

kubectl delete deployment taskboard
kubectl apply -f deployment.yaml
kubectl get pods
kubectl delete -f deployment.yaml
```

Note: A Deployment manages a **ReplicaSet**, which guarantees the desired Pod count is always running — this is the self-healing that naked Pods (Lab 13) lack.

✅ **Test it:** After `kubectl delete pod <name>`, `kubectl get pods` still shows 3 Pods because the Deployment recreated the missing one automatically.

Lab 16 — Rolling Updates & Rollbacks

Folder: `labs/lab16-rollouts/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day2-03-k8s-rollouts

Goal

Ship a new version of TaskBoard with zero downtime, then instantly roll back when something's wrong. You'll update the container image, track the rollout, and revert to a previous revision.

What you'll build

Roll a Deployment forward to a new image with no downtime, then roll it back.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Part A — Deploy v1, then update the image

```
kubectl create deployment taskboard --image=nginx:1.24 --replicas=3
kubectl rollout status deployment taskboard
kubectl describe deployment taskboard | grep Image
```

Update the image — Kubernetes shifts Pods gradually from the old to the new ReplicaSet (rolling update, the default strategy):

```
kubectl set image deployment/taskboard nginx=nginx:1.25
kubectl rollout status deployment taskboard
kubectl describe deployment taskboard | grep Image   # now 1.25
```

Part B — Roll back

```
kubectl rollout history deployment taskboard
kubectl rollout undo deployment taskboard            # back to 1.24
kubectl describe deployment taskboard | grep Image
kubectl rollout undo deployment taskboard --to-revision=2   # forward to 1.25
again
kubectl delete deployment taskboard
```

Note: Rolling updates keep old and new Pods running simultaneously so users never see downtime; a rollback is just a rollout to a previous revision.

✅ **Test it:** `kubectl describe deployment taskboard | grep Image` shows `nginx:1.25` after the update and `nginx:1.24` after `kubectl rollout undo`.

Lab 17 — Services — ClusterIP & NodePort

Folder: `labs/lab17-services/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day2-04-k8s-services

Goal

Give your Pods a stable address. Pod IPs change constantly; a Service provides one durable endpoint with built-in load balancing. You'll expose TaskBoard internally with ClusterIP and externally with NodePort, both imperatively and from YAML.

What you'll build

Expose a Deployment with ClusterIP (internal) and NodePort (external) Services.

Part A — ClusterIP (internal)

```
kubectl create deployment taskboard --image=nginx --replicas=2
kubectl expose deployment taskboard --port=80 --target-port=80
kubectl get service taskboard
kubectl get endpoints taskboard      # the Pod IPs behind the Service
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Part B — NodePort (external)

```
kubectl delete service taskboard
kubectl expose deployment taskboard \
  --type=NodePort --port=80 --target-port=80
kubectl get service taskboard         # note the 3xxxx port
```

Part C — Declarative (Service + Deployment YAML)

```
apiVersion: v1
kind: Service
metadata:
  name: taskboard-svc
spec:
  type: NodePort
  selector:
    app: taskboard
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080

kubectl delete deployment taskboard service taskboard
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get svc taskboard-svc          # NodePort 30080
kubectl delete -f service.yaml -f deployment.yaml
```

Note: ClusterIP is internal-only (service-to-service). NodePort opens a port on every node for outside access. In the cloud, LoadBalancer fronts NodePort with a real external IP.

✅ **Test it:** `kubectl get endpoints taskboard` lists 2 Pod IPs behind the Service, and the NodePort Service shows a port in the 30000–32767 range.

Lab 18 — Kubernetes Storage — emptyDir, PV & PVC

Folder: `labs/lab18-storage/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day2-05-k8s-storage-jobs

Goal

Persist data in Kubernetes. You'll share a scratch directory between two containers with `emptyDir`, then provision durable storage with a PersistentVolume and claim it with a PersistentVolumeClaim so data survives Pod deletion.

What you'll build

Share data with emptyDir; provision durable storage with a PV + PVC.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Part A — emptyDir (shared, temporary)

`emptyDir` is a scratch volume shared by all containers in a Pod and deleted with the Pod. Here a `writer` container writes a file a `reader` container reads:

```
apiVersion: v1
kind: Pod
metadata:
  name: shared-pod
spec:
  containers:
    - name: writer
      image: busybox
      command: ["sh", "-c", "echo 'hello from writer' > /data/message.txt &&
sleep 3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
    - name: reader
      image: busybox
      command: ["sh", "-c", "sleep 5 && cat /data/message.txt && sleep 3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
  volumes:
    - name: shared-data
      emptyDir: {}

kubectl apply -f emptydir-pod.yaml
kubectl exec shared-pod -c reader -- cat /data/message.txt
kubectl delete pod shared-pod      # emptyDir data is gone
```

Part B — PersistentVolume & Claim (durable)

A PV is cluster storage; a PVC requests a slice of it. Bind them, then mount the claim in a Pod:

```
kubectl apply -f pv.yaml
kubectl apply -f pvc.yaml
kubectl get pv,pvc                 # both should show Bound

kubectl apply -f pod-with-pvc.yaml
kubectl exec pvc-pod -- cat /data/file.txt
kubectl delete pod pvc-pod
kubectl apply -f pod-with-pvc.yaml # recreate...
kubectl exec pvc-pod -- cat /data/file.txt   # ...data is still there
kubectl delete pod pvc-pod
kubectl delete -f pvc.yaml -f pv.yaml
```

Note: This PV/PVC pattern is how a real TaskBoard Postgres database keeps its data in Kubernetes — storage outlives any individual Pod.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

✅ **Test it:** After deleting and recreating `pvc-pod`, `kubectl exec pvc-pod -- cat /data/file.txt` still prints `persistent data` — the PVC kept it across Pod deletion.

Lab 19 — Jobs & CronJobs — Batch and Scheduled Tasks

Folder: `labs/lab19-jobs-cronjobs/` · **KillerCoda:** https://killercoda.com/tertiary-labs/course/killercoda/day2-05-k8s-storage-jobs

Goal

Run work that finishes (unlike a web server that runs forever). A Job runs Pods to completion — perfect for a TaskBoard report — and a CronJob runs Jobs on a schedule, perfect for a nightly cleanup.

What you'll build

Run a batch Job (report) to completion and a scheduled CronJob (cleanup).

Part A — Job (run to completion)

```
kubectl create job hello --image=busybox -- echo 'Hello from a Job!'
kubectl get jobs
kubectl logs job/hello
kubectl delete job hello
```

Declarative — 3 completions, 2 running in parallel:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: taskboard-report
spec:
  completions: 3
  parallelism: 2
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: report
          image: busybox
          command: ["sh", "-c", "echo 'Generating TaskBoard report...'; sleep
5; echo done"]

kubectl apply -f job.yaml
kubectl get pods                 # up to 2 running at once
kubectl get job taskboard-report # COMPLETIONS climbs to 3/3
kubectl delete -f job.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Part B — CronJob (scheduled)

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: taskboard-cleanup
spec:
  schedule: "*/1 * * * *"     # every minute (demo); real cleanup might be
nightly
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: cleanup
              image: busybox
              command: ["sh", "-c", "echo 'Cleaning up old TaskBoard
sessions...'; date"]

kubectl apply -f cronjob.yaml
kubectl get cronjob taskboard-cleanup
# wait ~1 minute for the first Job to be created:
kubectl get jobs
kubectl logs job/<created-job-name>
kubectl delete -f cronjob.yaml
```

Note: Jobs use `restartPolicy: Never` and do **not** restart after success — unlike a Deployment, which keeps Pods running forever. CronJobs use standard cron syntax.

✅ **Test it:** `kubectl get job taskboard-report` reaches `3/3` completions, and after ~1 minute the CronJob has spawned at least one Job visible in `kubectl get jobs`.

Troubleshooting Cheat-Sheet

Symptom	Likely cause & fix
`port is already allocated`	Another container uses that host port — change `-p 8081:5000` or `docker rm -f` the other one.
`Cannot connect to the Docker daemon`	Docker isn't running — start Docker Desktop (or use KillerCoda).
`denied: requested access to the resource is denied` on push	Run `docker login` and tag as `<your-user>/taskboard:1.0` (Lab 9).
Web app can't reach Redis/DB by name	Containers must share a **custom** network (Lab 7) or be in the same Compose file; the host is the **service/container name**.
Pod stuck `Pending`	`kubectl describe pod <name>` — usually no schedulable node or an unbound PVC.
PVC stuck `Pending`	No PV matches the request — check `storage` size and `accessModes` (Lab 18).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Service returns nothing	The Service `selector` must match the Pod `labels`; check `kubectl get endpoints`.

Glossary

Term	Meaning
Image	A read-only package with everything needed to run an app; built from a Dockerfile.
Container	A running (or stopped) instance of an image — an isolated process.
Dockerfile	The text recipe of instructions Docker uses to build an image.
Layer	One filesystem change in an image; layers are cached and reused across builds.
CMD / ENTRYPOINT	Default command vs fixed executable for a container (Lab 5).
Named volume / Bind mount	Docker-managed persistent storage vs a mounted host folder (Lab 6).
Bridge network	A virtual network; a custom bridge adds DNS so containers reach each other by name (Lab 7).
Registry / Docker Hub	A store of images that clusters and users pull from (Lab 9).
Docker Compose	A tool to define and run multi-service apps from one YAML file (Labs 10–12).
Pod	The smallest Kubernetes unit — one or more containers sharing network & storage.
Deployment / ReplicaSet	Manages a desired number of identical Pods; self-heals and updates them.
Service (ClusterIP / NodePort)	A stable network endpoint load-balancing across Pods (Lab 17).
Namespace	A virtual cluster for isolating environments (Lab 14).
PV / PVC	PersistentVolume (storage) and PersistentVolumeClaim (a request for it) — Lab 18.
Job / CronJob	Run-to-completion work, and a scheduled Job (Lab 19).
kubectl	The Kubernetes command-line tool.

You're done — congratulations! You've taken one app from a single container all the way to a scalable Kubernetes deployment.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg